Pandas Cheat Sheet

Data Analysis Functions — Definition · Usage · Hint · Example

All examples use: df = pd.read_csv('flipkart_orders.csv')

1. Loading & Basic Inspection

pd.read_csv(filepath)

■ **Definition:** Reads a CSV file and returns a DataFrame. Most commonly used function to load tabular data into Pandas.

■ *Hint:* Always check shape and dtypes right after loading to spot issues early.

```
df = pd.read_csv('flipkart_orders.csv')
print(df.shape)   # (515, 21)
```

df.head(n)

■ **Definition:** Returns the first n rows of the DataFrame. Default n=5.

■ *Hint:* Use head() to quickly verify your data loaded correctly.

```
df.head(10)   # Shows first 10 rows
```

df.tail(n)

■ **Definition:** Returns the last n rows of the DataFrame. Default n=5.

■ *Hint:* Use tail() to check if the last rows of data look correct.

```
df.tail(5)    # Shows last 5 rows
```

df.shape

■ **Definition:** Returns a tuple (rows, columns) representing the dimensions of the DataFrame.

■ *Hint:* Always check shape after loading, filtering, or merging data.

```
df.shape      # (515, 21) → 515 rows, 21 columns
```

df.columns

■ **Definition:** Returns an Index object containing all column names of the DataFrame.

■ *Hint:* Use list(df.columns) to get a plain Python list of column names.

```
df.columns
# Index(['order_id','product_name','category','price',...], dtype='object')
```

df.dtypes

■ **Definition:** Returns the data type of each column in the DataFrame.

■ *Hint:* If a numeric column shows 'object', it may have dirty values needing cleaning.

```
df.dtypes
# order_id      object
# price        float64
# quantity       int64
```

df.info()

■ **Definition:** Prints a concise summary: column names, non-null counts, dtypes, and memory usage.

■ *Hint:* Best way to spot missing values at a glance — look for columns where non-null count < total rows.

```
df.info()
# RangeIndex: 515 entries
# age   490 non-null  float64   ← 25 missing!
```

df.describe()

■ **Definition:** Generates descriptive statistics: count, mean, std, min, 25%, 50%, 75%, max for numeric columns.

■ *Hint:* Use describe(include='all') to also include stats for string/categorical columns.

```
df.describe()
# price  count:515  mean:8420  std:22100  min:99  max:120000
df.describe(include='all')
```

2. Selecting & Filtering

df['column'] / df[['col1','col2']]

■ **Definition:** Selects a single column (returns Series) or multiple columns (returns DataFrame).

■ *Hint:* Use double brackets [[ ]] when selecting multiple columns to get a DataFrame.

```
df['price']                       # Series
df[['product_name','price','revenue']]  # DataFrame
```

df.loc[row_label, col_label]

■ **Definition:** Label-based selection. Selects rows and columns by their labels/names.

■ *Hint:* loc includes both start and end in slices. Use for label-based row/col selection.

```
df.loc[0:4, 'product_name':'price']   # rows 0-4, cols product_name to price
df.loc[df['category']=='Electronics']  # filter rows
```

df.iloc[row_index, col_index]

■ **Definition:** Integer position-based selection. Selects rows and columns by integer positions.

■ *Hint:* iloc excludes the end index (like Python slicing). Use for position-based access.

```
df.iloc[0:5, 0:4]    # first 5 rows, first 4 cols
df.iloc[10, 2]       # row 10, col 2 (single value)
```

df[boolean_condition]

■ **Definition:** Filters rows where the condition is True. Returns a subset DataFrame.

■ *Hint:* Combine multiple conditions using & (and), | (or), ~ (not).

```
df[df['revenue'] > 10000]
df[(df['category']=='Electronics') & (df['city']=='Mumbai')]
df[df['order_status'].isin(['Delivered','Shipped'])]
```

df.query('expression')

■ **Definition:** Filters rows using a string query expression. Cleaner syntax for complex conditions.

■ *Hint:* Use @ to reference external Python variables inside query strings.

```
df.query('revenue > 5000 and category == "Electronics"')
min_age = 30
df.query('age > @min_age')
```

df.isin(values)

■ **Definition:** Filters rows where column values are in a given list.

■ *Hint:* Use ~ df['col'].isin([...]) to exclude specific values.

```
df[df['payment_method'].isin(['UPI','Credit Card'])]
df[~df['order_status'].isin(['Cancelled','Returned'])]
```

3. Sorting & Reindexing

df.sort_values(by, ascending=True)

■ **Definition:** Sorts the DataFrame by one or more columns.

■ *Hint:* Pass a list to 'by' for multi-column sort. Use na_position='last' to push NaN to end.

```
df.sort_values('price', ascending=False)          # most expensive first
df.sort_values(['category','revenue'], ascending=[True,False])
```

df.sort_index(ascending=True)

■ **Definition:** Sorts the DataFrame by its row index.

■ *Hint:* Useful after filtering or concatenating data that may have shuffled index values.

```
df.sort_index()              # sort by index ascending
df.sort_index(ascending=False)  # sort by index descending
```

df.reset_index(drop=True)

■ **Definition:** Resets the row index to 0, 1, 2, ... Use drop=True to discard the old index.

■ *Hint:* Always use drop=True unless you need to keep the old index as a column.

```
df_sorted = df.sort_values('revenue', ascending=False)
df_sorted.reset_index(drop=True).head()
```

df.set_index(column)

■ **Definition:** Sets a column as the row index of the DataFrame.

■ *Hint:* Use reset_index() to convert the index back to a column later.

```
df.set_index('order_id')         # order_id becomes the index
df.set_index('order_id').loc['ORD123456']
```

4. Missing Values

df.isnull() / df.isna()

■ **Definition:** Returns a boolean DataFrame — True where values are missing (NaN/None).

■ *Hint:* Use df.isnull().sum() to count missing values per column.

```
df.isnull().sum()          # missing count per column
df.isnull().sum() / len(df) * 100   # missing %
```

df.notnull()

■ **Definition:** Returns a boolean DataFrame — True where values are NOT missing.

■ *Hint:* Use df[df['age'].notnull()] to filter rows where age is present.

```
df[df['age'].notnull()]    # only rows where age exists
df.notnull().sum()         # non-null count per column
```

df.dropna(how, axis, subset, thresh)

■ **Definition:** Drops rows or columns that contain missing values.

■ *Hint:* how='any' drops if ANY value missing; how='all' drops only if ALL missing.

```
df.dropna()                          # drop rows with any NaN
df.dropna(subset=['age','city'])     # only check these cols
df.dropna(thresh=15)                 # keep rows with >=15 non-null
df.dropna(axis=1, how='all')         # drop columns all NaN
```

df.fillna(value / method)

■ **Definition:** Fills missing values with a constant, mean, median, mode, or fill method.

■ *Hint:* Use method='ffill' for forward fill and method='bfill' for backward fill.

```
df['age'].fillna(df['age'].median())
df['city'].fillna('Unknown')
df['rating'].fillna(df.groupby('category')['rating'].transform('mean'))
df.fillna(method='ffill')    # forward fill
```

5. Duplicates

df.duplicated(subset, keep)

■ **Definition:** Returns a boolean Series — True for duplicate rows.

■ *Hint:* keep='first' marks all duplicates except first as True. keep=False marks ALL duplicates.

```
df.duplicated().sum()                    # total duplicate rows
df.duplicated(subset=['order_id']).sum()  # dupes by order_id
df[df.duplicated()]                       # show duplicate rows
```

df.drop_duplicates(subset, keep)

■ **Definition:** Removes duplicate rows from the DataFrame.

■ *Hint:* Always check shape before and after to confirm how many duplicates were removed.

```
print('Before:', df.shape)
df = df.drop_duplicates()
print('After:', df.shape)
df.drop_duplicates(subset=['order_id'], keep='first')
```

6. Column Operations & Transformations

df['new_col'] = expression

■ **Definition:** Adds a new column by assigning a computed expression. Most fundamental operation.

■ *Hint:* Use existing columns in the expression. Pandas applies it row-wise automatically.

```
df['revenue'] = df['discounted_price'] * df['quantity']
df['discount_amt'] = df['price'] - df['discounted_price']
df['is_high_value'] = df['revenue'] > 5000
```

df['col'].apply(func) / df.apply(func, axis)

■ **Definition:** Applies a function to each element of a column or across each row/column.

■ *Hint:* Use lambda for short functions. axis=1 applies function row-wise on full DataFrame.

```
df['price_cat'] = df['price'].apply(lambda x: 'Budget' if x<1000 else 'Premium')
df['revenue_check'] = df.apply(lambda row: row['discounted_price']*row['quantity'], axis=1)
```

df['col'].map(dict / func)

■ **Definition:** Maps values in a Series using a dictionary or function. Element-wise on a Series.

■ *Hint:* Great for encoding or replacing specific values. Returns NaN for unmapped values.

```
status_map = {'Delivered':'Done', 'Cancelled':'Fail', 'Returned':'Return'}
df['status_label'] = df['order_status'].map(status_map)
```

df.rename(columns={'old':'new'})

■ **Definition:** Renames columns by passing a dictionary of old:new names.

■ *Hint:* Use inplace=True or reassign to df to save changes.

```
df.rename(columns={'discounted_price':'final_price', 'discount_pct':'disc_%'})
```

df.drop(columns=['col1','col2'])

■ **Definition:** Removes specified columns (or rows) from the DataFrame.

■ *Hint:* Use axis=1 for columns, axis=0 for rows. Or use columns= parameter directly.

```
df.drop(columns=['review','product_id'])
df.drop([0, 5, 10], axis=0)    # drop rows by index
```

df['col'].astype(dtype)

■ **Definition:** Converts a column to a specified data type.

■ *Hint:* Convert 'object' columns to 'category' to save memory. Use errors='coerce' for safety.

```
df['age'] = df['age'].astype(int)
df['category'] = df['category'].astype('category')
df['order_date'] = pd.to_datetime(df['order_date'])
```

7. String Operations (str accessor)

df['col'].str.lower() / .str.upper() / .str.title()

■ **Definition:** Converts string values to lowercase, uppercase, or title case.

■ *Hint:* Always normalize strings before groupby or filtering to avoid mismatch ('Electronics' vs 'ELECTRONICS').

```
df['category'] = df['category'].str.lower()
df['customer_name'] = df['customer_name'].str.title()
df['order_status'] = df['order_status'].str.upper()
```

df['col'].str.strip() / .str.lstrip() / .str.rstrip()

■ **Definition:** Removes leading and/or trailing whitespace from string values.

■ *Hint:* Always strip strings after loading CSV data — invisible spaces cause filter failures.

```
df['city'] = df['city'].str.strip()
df['product_name'] = df['product_name'].str.strip()
```

df['col'].str.replace(old, new, regex=False)

■ **Definition:** Replaces occurrences of a substring or pattern in string values.

■ *Hint:* Set regex=True to use regular expressions for advanced replacement patterns.

```
df['order_status'].str.replace('Delivered','Completed')
df['revenue_str'].str.replace(',','').str.replace('$','')   # clean currency
```

df['col'].str.contains(pattern, na=False)

■ **Definition:** Returns boolean Series — True if the string contains the given pattern.

■ *Hint:* Always set na=False to handle NaN rows; otherwise they return NaN instead of False.

```
df[df['product_name'].str.contains('Pro', na=False)]
df[df['review'].str.contains('recommend', case=False, na=False)]
```

df['col'].str.split(sep, expand=False)

■ **Definition:** Splits each string value by a separator. expand=True returns separate columns.

■ *Hint:* Use expand=True to get each part as a separate column immediately.

```
df['order_id'].str.split('D', expand=True)        # splits 'ORD123' into parts
df['full_name'].str.split(' ', expand=True)         # first / last name
```

df['col'].str.startswith() / .str.endswith()

■ **Definition:** Returns boolean Series — True if the string starts/ends with the given prefix/suffix.

■ *Hint:* Useful for filtering product codes, order IDs, or any prefixed identifiers.

```
df[df['customer_name'].str.startswith('R', na=False)]
df[df['order_id'].str.startswith('ORD', na=False)]
```

df['col'].str.len()

■ **Definition:** Returns the length of each string value in the Series.

■ *Hint:* Useful for data validation — e.g. check if order_id always has expected length.

```
df['customer_name'].str.len().describe()
df[df['order_id'].str.len() != 9]   # detect malformed IDs
```

8. Date-Time Operations (dt accessor)

pd.to_datetime(series, format=None, errors='coerce')

■ **Definition:** Converts a string column to datetime format. Essential before using .dt accessor.

■ *Hint:* Use errors='coerce' to convert invalid date strings to NaT instead of raising error.

```
df['order_date'] = pd.to_datetime(df['order_date'])
df['delivery_date'] = pd.to_datetime(df['delivery_date'], errors='coerce')
```

df['col'].dt.year / .dt.month / .dt.day

■ **Definition:** Extracts year, month, or day as an integer from a datetime column.

■ *Hint:* Must convert to datetime first with pd.to_datetime() before using .dt accessor.

```
df['order_year']  = df['order_date'].dt.year
df['order_month'] = df['order_date'].dt.month
df['order_day']   = df['order_date'].dt.day
```

df['col'].dt.day_name() / .dt.month_name()

■ **Definition:** Returns the day name ('Monday') or month name ('January') for each datetime value.

■ *Hint:* Use value_counts() on day_name() to find which day has most orders.

```
df['order_date'].dt.day_name().value_counts()
df['order_date'].dt.month_name().value_counts()
```

datetime subtraction / pd.Timedelta

■ **Definition:** Calculates the difference between two datetime columns as a timedelta.

■ *Hint:* Use .dt.days to extract integer number of days from a timedelta Series.

```
df['delivery_time'] = (pd.to_datetime(df['delivery_date'])
                     - pd.to_datetime(df['order_date'])).dt.days
df[df['delivery_time'] > 7]   # slow deliveries
```

df[df['date_col'].dt.year == 2023]

■ **Definition:** Filters rows based on year, month, quarter, or any datetime component.

■ *Hint:* Chain multiple .dt filters with & to create date range filters easily.

```
df[df['order_date'].dt.year == 2023]
df[(df['order_date'].dt.year==2023) & (df['order_date'].dt.month<=3)]
```

pd.date_range(start, end, freq)

■ **Definition:** Generates a sequence of dates between start and end with given frequency.

■ *Hint:* Useful for creating date indexes, reindexing time series, or filling missing dates.

```
pd.date_range('2023-01-01', '2023-12-31', freq='M')  # month-end dates
pd.date_range(start='2022-01-01', periods=12, freq='MS') # 12 month-starts
```

9. GroupBy & Aggregation

df.groupby('col').agg_func()

■ **Definition:** Groups data by a column and applies an aggregation function to each group.

■ *Hint:* groupby returns a GroupBy object — always chain with an aggregation function.

```
df.groupby('category')['revenue'].sum()
df.groupby('city')['rating'].mean()
df.groupby('order_status')['order_id'].count()
```

df.groupby('col').agg({'col1':'sum', 'col2':'mean'})

■ **Definition:** Applies different aggregation functions to different columns simultaneously.

■ *Hint:* Use named aggregations with .agg(alias=('column','func')) for cleaner output.

```
df.groupby('category').agg({'revenue':'sum','rating':'mean','order_id':'count'})
df.groupby('category').agg(total_rev=('revenue','sum'), avg_rating=('rating','mean'))
```

df.groupby(['col1','col2']).func()

■ **Definition:** Groups by multiple columns to create a hierarchical (multi-level) grouping.

■ *Hint:* Use .reset_index() after groupby to convert the result back to a flat DataFrame.

```
df.groupby(['category','city'])['revenue'].sum().reset_index()
df.groupby(['category','order_status'])['order_id'].count().reset_index()
```

df.groupby('col').transform('func')

■ **Definition:** Like groupby but returns a Series with the same index as original df — for adding group-level stats back.

■ *Hint:* Use transform when you want to fill missing values with group mean/median.

```
# Fill missing age with mean age per Pclass
df['age'] = df['age'].fillna(df.groupby('category')['age'].transform('mean'))
df['group_mean_revenue'] = df.groupby('category')['revenue'].transform('mean')
```

df.pivot_table(values, index, columns, aggfunc)

■ **Definition:** Creates a spreadsheet-style pivot table from a DataFrame.

■ *Hint:* aggfunc default is 'mean'. Use fill_value=0 to replace NaN in pivot output.

```
df.pivot_table(values='revenue', index='category',
               columns='order_status', aggfunc='sum', fill_value=0)
```

df['col'].value_counts(normalize=False)

■ **Definition:** Counts how many times each unique value appears in a column.

■ *Hint:* Use normalize=True to get proportions (0–1) instead of raw counts.

```
df['category'].value_counts()          # count per category
df['payment_method'].value_counts(normalize=True)  # % share
```

df.nunique() / df['col'].unique()

■ **Definition:** nunique() counts distinct values per column. unique() returns an array of distinct values.

■ *Hint:* nunique() is great for detecting high-cardinality columns before encoding.

```
df.nunique()                  # distinct values per column
df['city'].unique()           # array of all unique cities
df['category'].nunique()      # → 6
```

10. Merging & Joining DataFrames

pd.merge(df1, df2, on='key', how='inner')

■ **Definition:** Merges two DataFrames on a common column (key). Similar to SQL JOIN.

■ *Hint:* how options: 'inner' (common only), 'left', 'right', 'outer' (all rows).

```
pd.merge(df_orders, df_customers, on='customer_id', how='inner')
pd.merge(df1, df2, on=['customer_id','product_id'], how='left')
```

pd.concat([df1, df2], axis=0)

■ **Definition:** Stacks DataFrames vertically (axis=0) or horizontally (axis=1).

■ *Hint:* Use ignore_index=True to reset index after concatenation.

```
pd.concat([df_north, df_south], ignore_index=True)       # stack rows
pd.concat([df_info, df_orders], axis=1)                   # stack columns
```

df1.join(df2, on='key', how='left')

■ **Definition:** Joins two DataFrames on their index or a key column. Simpler syntax than merge.

■ *Hint:* join() works on index by default. Use merge() for column-based joins.

```
df1.set_index('customer_id').join(df2.set_index('customer_id'), how='left')
```

11. Outlier Detection

IQR Method

■ **Definition:** Interquartile Range: values below Q1-1.5*IQR or above Q3+1.5*IQR are outliers.

■ *Hint:* IQR method is robust to extreme skew. Use it as your default outlier detection.

```
Q1  = df['revenue'].quantile(0.25)
Q3  = df['revenue'].quantile(0.75)
IQR = Q3 - Q1
outliers = df[(df['revenue'] < Q1-1.5*IQR) | (df['revenue'] > Q3+1.5*IQR)]
```

Z-Score Method

■ **Definition:** Values with absolute Z-score > 3 are considered outliers (more than 3 std devs from mean).

■ *Hint:* Z-score works best when data is roughly normal. IQR is better for skewed data.

```
from scipy import stats
z_scores = stats.zscore(df['revenue'].dropna())
outliers  = df[abs(stats.zscore(df['revenue'].fillna(df['revenue'].mean()))) > 3]
```

Capping (Winsorization)

■ **Definition:** Replaces outlier values with the lower/upper percentile boundary instead of removing them.

■ *Hint:* Use clip() to cap values between lower and upper bounds cleanly in one line.

```
lower = df['revenue'].quantile(0.05)
upper = df['revenue'].quantile(0.95)
df['revenue'] = df['revenue'].clip(lower=lower, upper=upper)
```

12. Quick Reference Summary

Function	Purpose	Key Param
`df.head(n) / tail(n)`	First/last n rows	n=5
`df.info()`	Column types + null counts	—
`df.describe()`	Stats summary	include='all'
`df.shape / .columns`	Dimensions / column names	—
`df.loc[ ]`	Label-based selection	row, col labels
`df.iloc[ ]`	Position-based selection	row, col integers
`df.sort_values()`	Sort by column	by, ascending
`df.reset_index()`	Reset row index	drop=True
`df.isnull().sum()`	Count missing per column	—
`df.dropna()`	Drop rows/cols with NaN	how, subset, thresh
`df.fillna()`	Fill missing values	value, method
`df.drop_duplicates()`	Remove duplicate rows	subset, keep
`df['col'].apply()`	Apply function element-wise	lambda, func
`df['col'].map()`	Map values via dict/func	dict / function
`df.rename()`	Rename columns	columns={'old':'new'}
`df.astype()`	Convert column dtype	int, float, category
`df['col'].str.*`	String operations	lower, replace, split
`pd.to_datetime()`	Convert to datetime	errors='coerce'
`df['col'].dt.*`	DateTime extraction	year, month, day
`df.groupby().agg()`	Group + aggregate	sum, mean, count
`df.pivot_table()`	Pivot summary table	values, index, aggfunc
`df.value_counts()`	Count unique values	normalize=True
`pd.merge()`	Join two DataFrames	on, how
`pd.concat()`	Stack DataFrames	axis, ignore_index
`df.clip(lower, upper)`	Cap outlier values	lower, upper

Pandas Data Analysis Cheat Sheet | All Functions with Definition · Hint · Example